



ZHONYAL

Documentación de la Práctica — Interfaz

Aplicación de Gestión de Estudio

Asignatura	Sistemas Interactivos
Curso	2025/26
Autor	Javier Castilla Arroyo - i32casaj
Plataforma	Escritorio PC (1353x929px)
Repositorio	github.com/JxViii/Zhonyal

Índice

1. Implementación de la interfaz.....	3
1.1. Stack tecnológico y dependencias.....	3
1.2. Organización del código.....	3
1.3. Aspectos significativos de la implementación.....	6
1.3.1. Enrutado mediante CardLayout.....	6
1.3.2. Modelo de sesiones y splits.....	6
1.3.3. Filtrado de sesiones.....	7
1.3.4. Persistencia local con SQLite.....	7
1.3.5. Visor de PDF integrado (PDFBox).....	7
1.3.6. Cronómetro flotante (Chrono).....	7
1.3.7. Componentes fieles al diseño.....	8
1.3.8. Perfil local y validación de formularios.....	8
1.3.9. Casos de PopUp de advertencia.....	9
1.4. Internacionalización (i18n).....	9
1.5. Conceptos teóricos aplicados.....	9
1.6. Diferencias respecto al prototipo.....	10
2. Tutorial / Manual de usuario.....	12
2.1. Requisitos.....	12
2.2. Compilación y ejecución.....	12
2.3. Recorrido por la interfaz.....	12
Onboarding.....	12
Inicio (Home).....	13
Sesiones.....	14
Cronómetro flotante.....	15
Historial de sesiones.....	15
Detalle de sesión.....	15
Notas.....	16
Visor de documentos.....	17
Idioma y menú lateral.....	17
3. Conclusiones.....	19
3.1 Cosas que mejorar y añadir.....	19
3.2 Uso de la IA.....	19

1. Implementación de la interfaz

Esta entrega materializa el prototipo presentado en la fase anterior como una aplicación de escritorio funcional desarrollada en **Java con la biblioteca Swing**. Zhonyal permite al estudiante cronometrar sesiones de estudio distinguiendo tiempo activo y de pausa mediante *splits*, consultar el historial y el detalle de cada sesión con su *focus rate*, y gestionar y visualizar sus apuntes en PDF sin salir de la aplicación. Toda la interfaz está internacionalizada en español e inglés con cambio de idioma en caliente.

1.1. Stack tecnológico y dependencias

La interfaz se ha construido íntegramente con componentes Swing, apoyándose en un conjunto reducido de dependencias externas que se empaquetan junto al código en un único JAR ejecutable (fat-JAR):

Dependencia	Versión	Propósito
Java Swing	—	Construcción de toda la interfaz gráfica (ventanas, componentes y pintado a medida).
sqlite-jdbc	3.46.0	Base de datos local SQLite para sesiones y splits.
Apache PDFBox	3.0.7	Renderizado de documentos PDF en el visor integrado.
SLF4J	2.0.16	Fachada de logging (configurada como no-op).
ResourceBundle	JDK	Internacionalización mediante ficheros de propiedades.

Para **ejecutar** la aplicación basta con un JRE de Java 17 o superior; para **compilar desde el código fuente** se utiliza JDK 25. La aplicación se distribuye como JAR autocontenido (en Windows, adicionalmente, como instalador con JRE incluido mediante jpackage).

1.2. Organización del código

El proyecto se estructura por **capas**, separando el modelo de dominio, la vista y la infraestructura. La clase `App` actúa como controlador y enrutador central, mientras que el paquete `ui` se subdivide en páginas completas, componentes reutilizables y plantillas del armazón de la aplicación.



Figura 1. Estructura de directorios del proyecto.

La capa de modelo y utilidades reside en `helpers/` (las entidades `Session`, `Split` y `Type`, junto a `LangManager`, `FilterState`, `UserProfile` y utilidades de formato y ficheros). La vista vive en `ui/`, repartida en `pages/` (una clase por pantalla), `components/` (componentes agrupados por página correspondiente) y `templates/` (el `AppShell`, la cabecera y el menú lateral). Finalmente, `utils/` concentra la infraestructura: acceso a base de datos (DB), paleta (Theme) y carga de tipografías (Fonts).

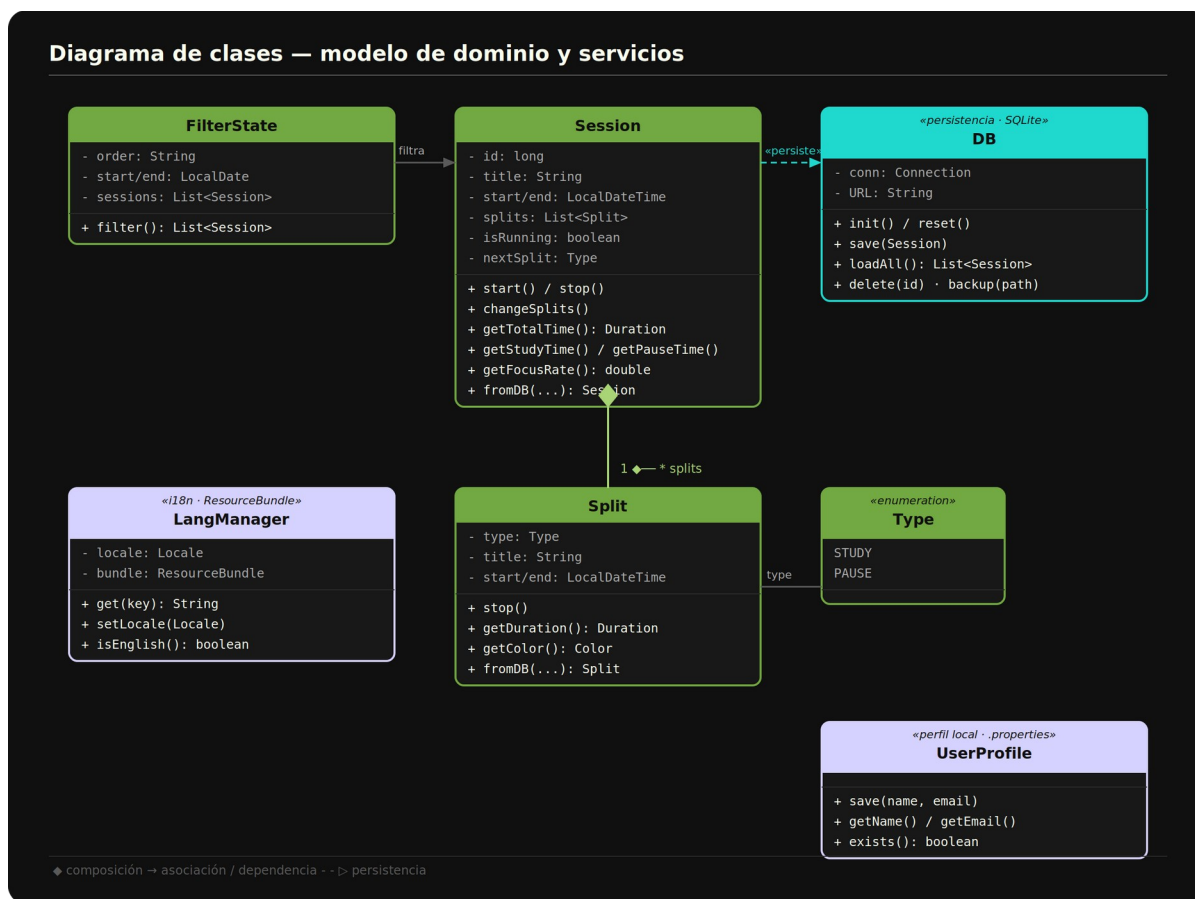


Figura 2. Diagrama de clases: modelo de dominio y servicios.

El núcleo del dominio es la relación de **composición** entre `Session` y `Split`: una sesión agrega una lista ordenada de tramos, cada uno de tipo `STUDY` o `PAUSE` (enumerado `Type`). La persistencia se delega en `DB` (`SQLite`), que reconstruye los objetos mediante métodos de fábrica `fromDB(...)`. `FilterState` ordena y filtra el historial, y los servicios `LangManager` (`i18n`) y `UserProfile` (perfil local) dan soporte transversal a la interfaz.

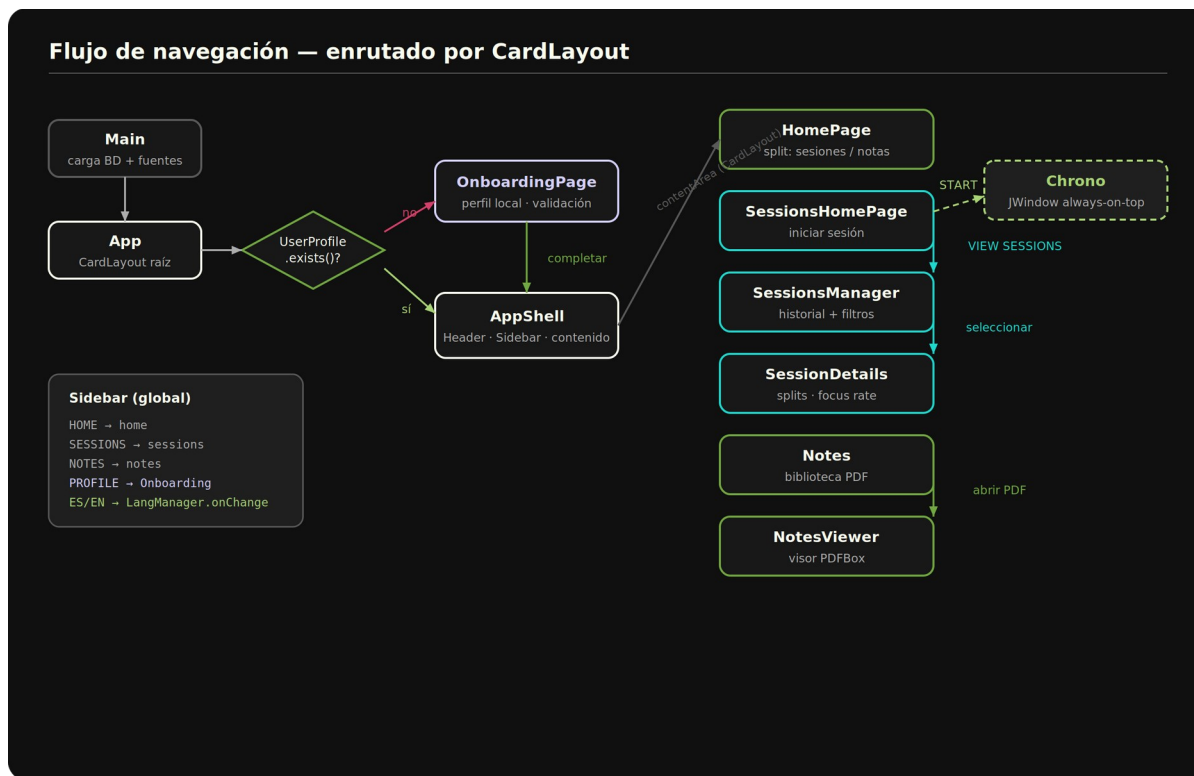


Figura 3. Flujo de navegación: enrutado por CardLayout.

La navegación se resuelve con CardLayout en dos niveles. El nivel raíz, en App, alterna entre el *Onboarding* y el AppShell según si existe un perfil (`UserProfile.exists()`). Dentro del AppShell, un segundo CardLayout intercambia las seis vistas de contenido. El cronómetro es la excepción: se materializa como una ventana JWindow independiente y flotante, fuera del flujo de tarjetas.

1.3. Aspectos significativos de la implementación

1.3.1. Enrutado mediante CardLayout

Cada cambio de pantalla pasa por `App.navigate(...)` o por el menú lateral. Algunas tarjetas (el gestor de sesiones y el detalle) se **reconstruyen** en cada navegación con `replaceCard(...)`, de modo que siempre reflejen los datos más recientes de la base de datos y el idioma activo. La cabecera, el menú y el fondo cambian según el tipo de página (Page «Sessions» en cian frente a «Default» en verde), reforzando el contexto visual.

1.3.2. Modelo de sesiones y splits

Al iniciar una sesión se crea el primer split de estudio; el botón central del cronómetro invoca `changeSplits()`, que cierra el tramo actual y abre uno del tipo contrario, alternando estudio y pausa. Las duraciones se calculan con `Duration.between(...)` sobre marcas `LocalDateTime`; para el tramo en curso se usa la hora actual, lo que permite un cronómetro vivo. El *focus rate* se obtiene como el cociente entre tiempo de estudio y tiempo total.

1.3.3. Filtrado de sesiones

Para el filtrado de sesiones nos hemos decantado por filtrar respecto a fechas, en vez de implementar tags.

Para esta tarea, creamos `FilterState`, la idea para esta clase, viene de un proyecto personal en el que tenía que filtrar entradas de un blog, y acabe optamos por este estilo de filtrado. Básicamente tenemos un objeto de esta clase, que guarda el orden en el que se tiene que filtrar (más antiguo, más nuevo), y una fecha de inicio y de final como límite para el filtrado.

1.3.4. Persistencia local con SQLite

La clase DB crea dos tablas relacionadas (sessions y splits, con clave foránea) y guarda cada sesión de forma **transaccional**: desactiva el *autocommit*, inserta la sesión y sus splits por lotes, confirma con commit y revierte con rollback ante cualquier error. La ruta de la base de datos se adapta al sistema operativo (en Windows se ubica bajo LOCALAPPDATA). El historial se recupera con loadAll() y se ofrece borrado por sesión y un respaldo con VACUUM INTO.

1.3.5. Visor de PDF integrado (PDFBox)

El visor divide la pantalla en una lista de apuntes y un panel de documento. Cada página se rasteriza con PDFRenderer a 150 DPI dentro de un SwingWorker, de forma que el renderizado pesado ocurre **fuera del hilo de eventos** (EDT) y la interfaz no se congela. Al cargar un documento se ajusta automáticamente al ancho disponible y se ofrece zoom entre el 30 % y el 200 %. Los apuntes se almacenan localmente en `~/zhonyal/files`.

1.3.6. Cronómetro flotante (Chrono)

El cronómetro es una ventana JWindow siempre visible por encima del resto (setAlwaysOnTop), con esquinas redondeadas (setShape) y arrastrable mediante escuchadores de ratón. Un Timer de un segundo actualiza el tiempo total, el tiempo del tramo en curso y la hora actual. El botón central alterna estudio/pausa y recolorea el indicador (verde/rojo); el botón izquierdo **colapsa** la ventana dejando solo los controles (442×124 px); el botón derecho detiene la sesión, la persiste y cierra la ventana.

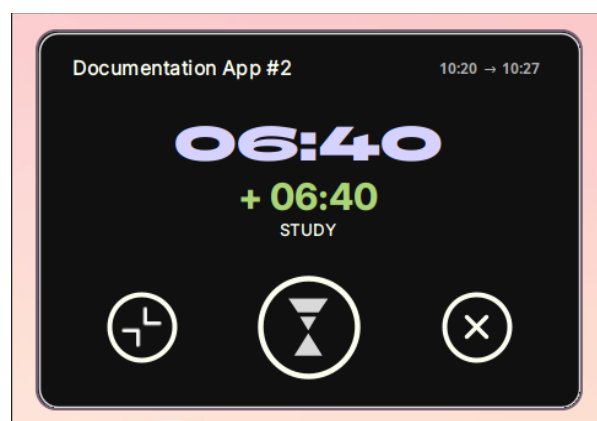


Figure 3: Chrono - Study Split

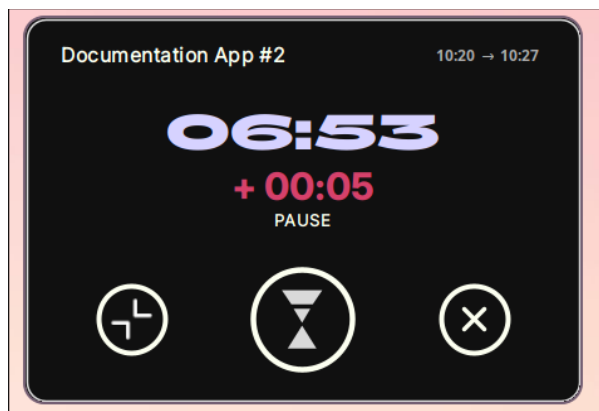


Figura 4. Chrono – Pause Split



Figura 5. Chrono colapsado: solo los controles permanecen visibles.

1.3.7. Componentes fieles al diseño

Este es un punto bastante importante que quería tratar. En el prototipado hice un diseño relativamente ambicioso, ¿a qué me refiero? Me refiero a que diseñe la aplicación como lo haría con **HTML** y **CSS**, esperando que me faltase algo que aprender de **Java Swing** que me hiciese la vida exponencialmente más fácil a la hora de diseñar.

Ese aprendizaje no llego, y tuve que empezar a diseñar desde 0, teniendo que descargarme fuentes, importarlas y probar mil configuraciones para que estas cargasen bien, hasta aprender a usar `paintComponent`, ya que **Java Swing** no tiene forma de crear bordes curvos, cosa que no se me había venido a la cabeza a la hora de crear un diseño de aplicación sin un solo borde recto.

Al final le acabé pillando el truco a todo, pero las primeras páginas me pudieron costar días, igual para el primer componente que hice (`HomeCard.java`), pude estar unas 10 horas, entre importar colores, fuentes, funciones y cuadrar todo.

1.3.8. Perfil local y validación de formularios

En el primer arranque, la pantalla de *Onboarding* actúa como puerta de entrada: pide nombre y correo, valida que ningún campo quede vacío y comprueba el formato del email con una expresión regular, mostrando mensajes de error en rojo bajo los campos. El perfil se guarda en `~/zhonyal/user.properties` y puede editarse después desde la opción «Perfil» del menú lateral.

Como teníamos que crear un formulario, no quería desechar la idea de un log-in, pero tampoco quería que la aplicación necesitase de ser online para funcionar, ni meter mucha complejidad, para lo que creo que es un concepto muy sencillo. Entonces he optado por meter un log-in

testimonial, que crea tu username, y que pide un e-mail para funcionalidades que crearé con fines propios en un futuro.

1.3.9. Casos de PopUp de advertencia

Algo que no había pensado en el prototipado eran los mensajes de advertencia en ciertas acciones, por lo que he creado este apartado para poner los casos en los que aparecen:

- **Crear una sesión sin título:** Advertencia de que se debe de introducir un título
- **Crear una sesión con otra activa:** Solo se puede tener una sesión activa
- **Cerrar la app con una sesión activa:** ¿Estás seguro de cerrar la app? La sesión activa se perderá
- **Eliminar sesiones o archivos:** ¿Estás seguro de borrar <archivo>? Si o No

1.4. Internacionalización (i18n)

La internacionalización es uno de los aspectos centrales de la entrega. LangManager envuelve un ResourceBundle: toma el idioma inicial del sistema, resuelve cada texto por clave con get(key) (con repliegue seguro a la propia clave si faltara) y expone setLocale(...) junto a un callback onChange que repinta la interfaz. El botón ES/EN del menú lateral cambia el idioma **en caliente**; App aprovecha ese callback para refrescar etiquetas y reconstruir las tarjetas dinámicas (gestor y detalle de sesión), de modo que incluso las vistas generadas al vuelo cambian de idioma sin reiniciar.

Los textos se mantienen en dos ficheros de propiedades. A modo de muestra:

Clave	Bundle_en	Bundle_es
sidebar.home	HOME	INICIO
sidebar.sessions	SESSIONS	SESIONES
sidebar.reset	RESET	LIMPIAR
onboarding.button	Create an account	Crear una cuenta
session.details.focus	FOCUS RATE	CONCENTRACIÓN
split.study / split.pause	STUDY / PAUSE	ESTUDIO / PAUSA

Algunas etiquetas multilínea aprovechan fragmentos HTML admitidos por Swing (por ejemplo <html>Your
Sessions</html>). La Figura 11, al final del manual, muestra la pantalla de inicio en español.

1.5. Conceptos teóricos aplicados

Java-Swing: Voy a meter esto como concepto teórico, ya que al final es lo que hemos dado en la parte de prácticas de la asignatura. He utilizado todo lo usable dentro de Swing, aprendido todo lo relativo al diseño de los componentes y formas de hacer cosas más fáciles, como

`addActionListener( e -> {} )`, por ejemplo.

Consistencia en el diseño: La interfaz que he creado tiene un diseño bastante homogéneo, en el que se distinguen claramente colores para ambas partes de la app: sesiones y apuntes. No solo eso, sino que también solo usamos 4 fuentes de cara al diseño de la app:

- **BBH Bartle:** Usada para los títulos principales
- **BBH Hegarty:** Usada para algunos componentes como los items del Sidebar
- **Inter:** Fuente principal en la app, se usan 2 variantes, la regular, y la extraBold.
- **Gochi Hand:** Esta se usa para los detalles de fuente que parezca que están escritas a mano, no pude encontrar la misma que use en Figma para esto, y opte por esta alternativa.

Programa sencillo: Mi aplicación cumple esto a la perfección, estamos hablando de un app offline, que es fácil de entender y rápida de iniciar, para que no pierdas tiempo de hacer otras cosas por tener que usar está.

Internacionalización: No me voy a parar mucho aquí, porque esto era un requisito del trabajo. He conseguido que la aplicación esté en dos idiomas: español e inglés, y que se pueda cambiar en cualquier momento, en cualquier página.

Feedback: Para poder crear una aplicación para mi público objetivo, estudiantes de mi perfil principalmente, he de pedir ayuda y opiniones respecto al diseño, y a las funcionalidades de la aplicación, como empecé con la app prácticamente el día después de entregar el prototipado, pude pedir y recibir mucho feedback de compañeros y conocidos.

1.6. Diferencias respecto al prototipo

Tal y como pide el enunciado, se detallan a continuación las diferencias relevantes entre el prototipo de Figma y la interfaz final, todas ellas justificadas. La mayoría responden a la adaptación de un *mockup* de mentalidad web a una aplicación de escritorio nativa.

Elemento del prototipo	Implementación final y justificación
Login / Registro / «Skip Log In»	Sustituidos por una única pantalla de Onboarding local. Al ser una aplicación de escritorio monousuario y sin servidor, no hay credenciales que autenticar: solo se personaliza el perfil. El estado de error «credenciales inválidas» pasa a ser validación de formulario (campos vacíos y formato de email).
Página de error 404	Eliminada. En una aplicación de escritorio con CardLayout no existen rutas ni URLs a las que un usuario pueda acceder erróneamente, por lo que la vista carece de sentido.
Sección de Métricas (gráfica semanal/mensual y racha)	No implementada como pantalla independiente por motivos de alcance y tiempo. No obstante, la métrica central —el focus rate— y el desglose estudio/pausa sí están presentes a nivel de cada sesión en el Detalle de Sesión.
Filtros por etiquetas	Reemplazados por filtro de orden (más nuevas / más antiguas) y por rango de fechas. El modelo de sesiones actual no maneja etiquetas,

	y el filtrado temporal cubre el caso de uso principal del historial.
«LOG OUT»	Reconvertido en «RESET / LIMPIAR» (borra los apuntes locales) y se añade la entrada «Perfil». Sin login no hay cierre de sesión, de modo que el espacio se reutiliza para la gestión de datos locales y la edición del perfil.
Cambio de idioma por botón	En el prototipo figuraba como pendiente; en la interfaz final está plenamente implementado con cambio en caliente ES/EN.
Cronómetro flotante	Del cronómetro flotante simplemente hemos cambiado el botón de añadir archivo, a un botón que colapsa el cronómetro para que se pueda ocultar un poco si el usuario lo desea.

2. Tutorial / Manual de usuario

2.1. Requisitos

- **Para ejecutar:** Java 17 o superior (JRE).
- **Para compilar desde el código fuente:** JDK 25.

2.2. Compilación y ejecución

El paquete entregado contiene el código fuente en `codigo/`, las dependencias en `lib/`, los recursos en `images/` y `fonts/`, y el JAR ejecutable `zhonyal.jar`. La forma más rápida de ejecutar la aplicación es situarse en la carpeta del entregable (la que contiene `images/` y `fonts/`, ya que los recursos se cargan por ruta relativa) y lanzar el JAR:

```
cd nombre_del_entregable/  
java -jar zhonyal.jar
```

Para **compilar desde el código fuente** en Linux, el repositorio incluye un script que compila y genera el fat-JAR en `dist/`:

```
bash build.sh  
java -jar dist/zhonyal.jar
```

Alternativamente, la compilación manual equivalente sería:

```
javac -cp "lib/*" -d out -sourcepath codigo codigo/Main.java  
cp -r codigo/i18n out/  
java -cp "out:lib/*" Main # ejecutar donde estén images/ y fonts/
```

En Windows se utiliza `build.bat` de forma análoga. El repositorio incluye además `install.sh` (instalación en el menú de aplicaciones de Linux) y los scripts `package.sh` / `package-windows.bat` para generar paquetes de distribución (este último, un instalador con JRE incluido vía `jpackage`).

Para Windows recomendable ir [aquí](#), a descargar el instalador.

2.3. Recorrido por la interfaz

Onboarding

Pantalla de bienvenida en el primer arranque. Solicita un nombre de perfil y un email, con validación de campos vacíos y de formato de correo, y crea el perfil local.



Figura 6. Pantalla de Onboarding.

Inicio (Home)

Pantalla principal tras crear el perfil. Muestra un saludo personalizado y dos accesos directos en pantalla dividida: Sesiones a la izquierda y Notas a la derecha. El menú lateral se abre desde el botón de la cabecera.

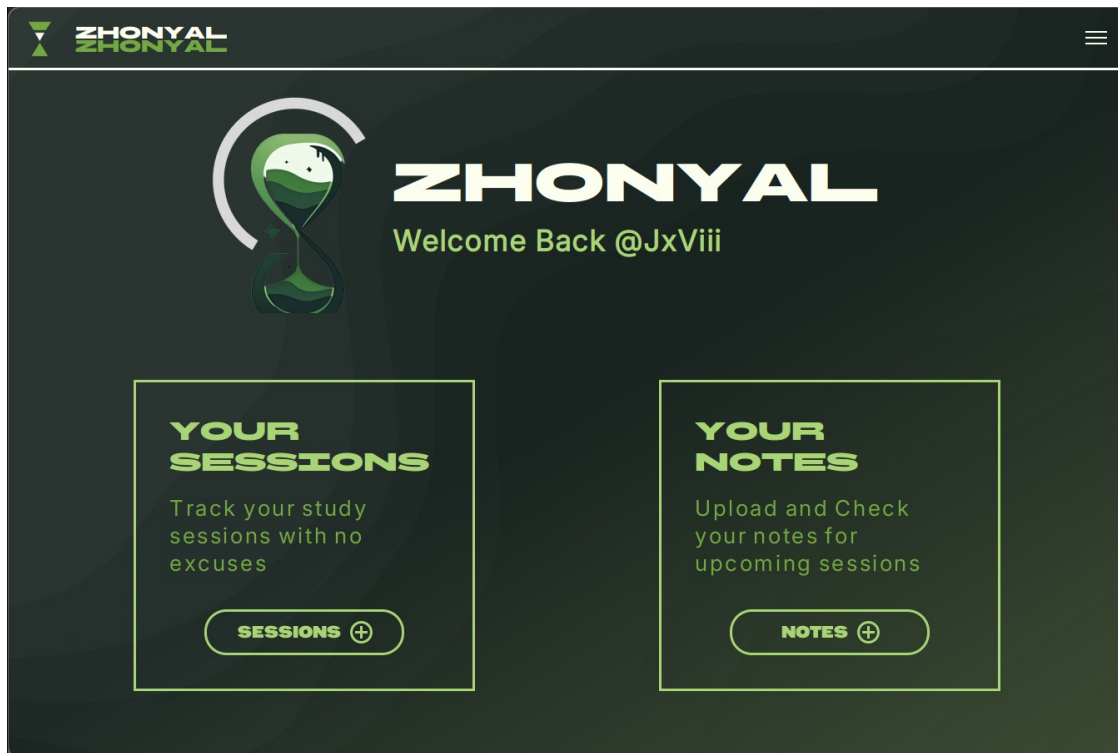


Figura 7. Pantalla de Inicio.

Sesiones

Punto de partida de la sección de sesiones: se introduce un título y se inicia una nueva sesión cronometrada, o se accede al historial mediante «View Sessions».

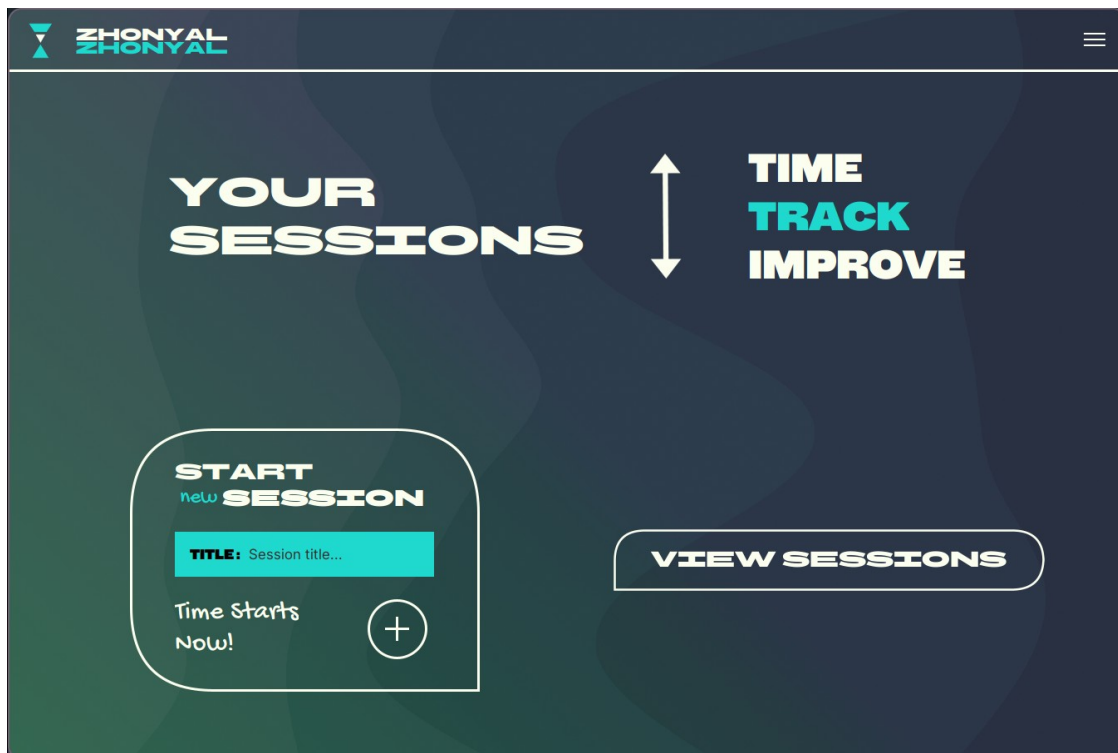


Figura 8. Pantalla de Sesiones.

Cronómetro flotante

Al iniciar una sesión aparece el cronómetro como ventana flotante siempre visible. Distingue tiempo de estudio (verde) y de pausa (rojo), puede colapsarse para ocupar menos espacio y se detiene guardando la sesión.

El tiempo sigue corriendo hasta que se cierre la sesión, y cambias de modo presionando el botón central del cronómetro.

Las capturas correspondientes se muestran en las Figuras 4 y 5.

Historial de sesiones

Listado de todas las sesiones registradas, con su tiempo total, de estudio y de pausa, y fechas de inicio y fin. Incluye una barra de filtros por orden (más nuevas / más antiguas) y por rango de fechas, además del modo de borrado.

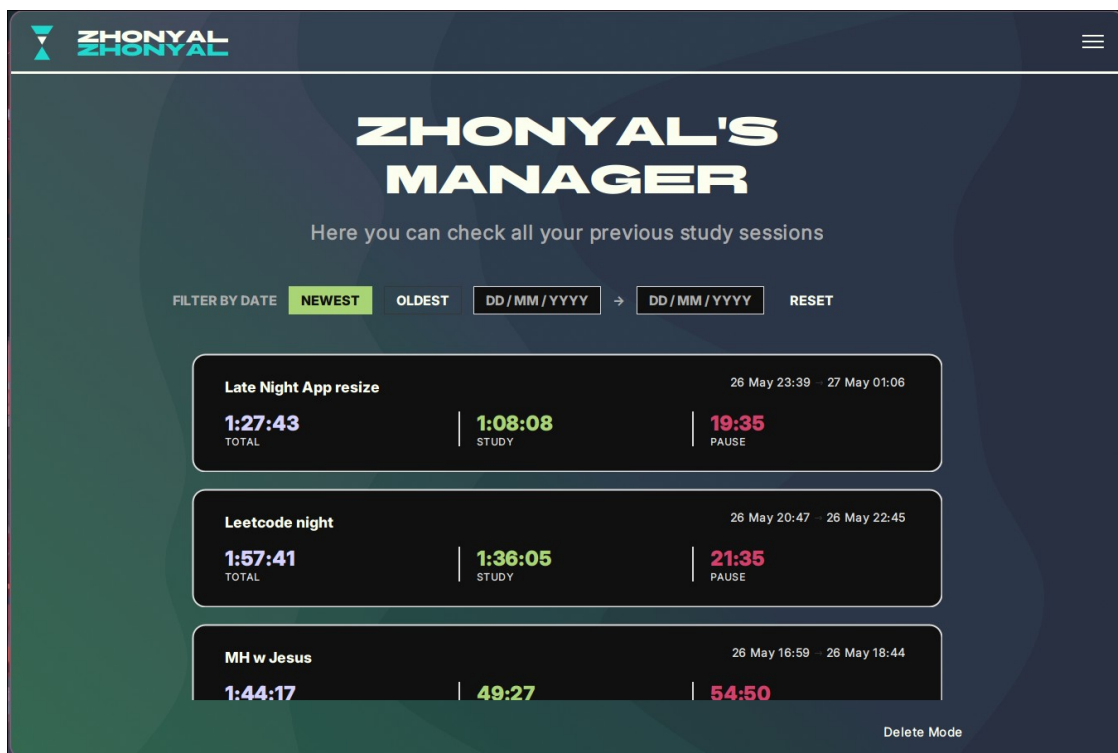


Figura 9. Historial de sesiones (Zhonyal's Manager).

Detalle de sesión

Vista detallada de una sesión: datos globales (inicio, fin y tiempos totales) y desglose de todos los splits con su duración individual, diferenciando estudio y pausa, junto al focus rate de la sesión.

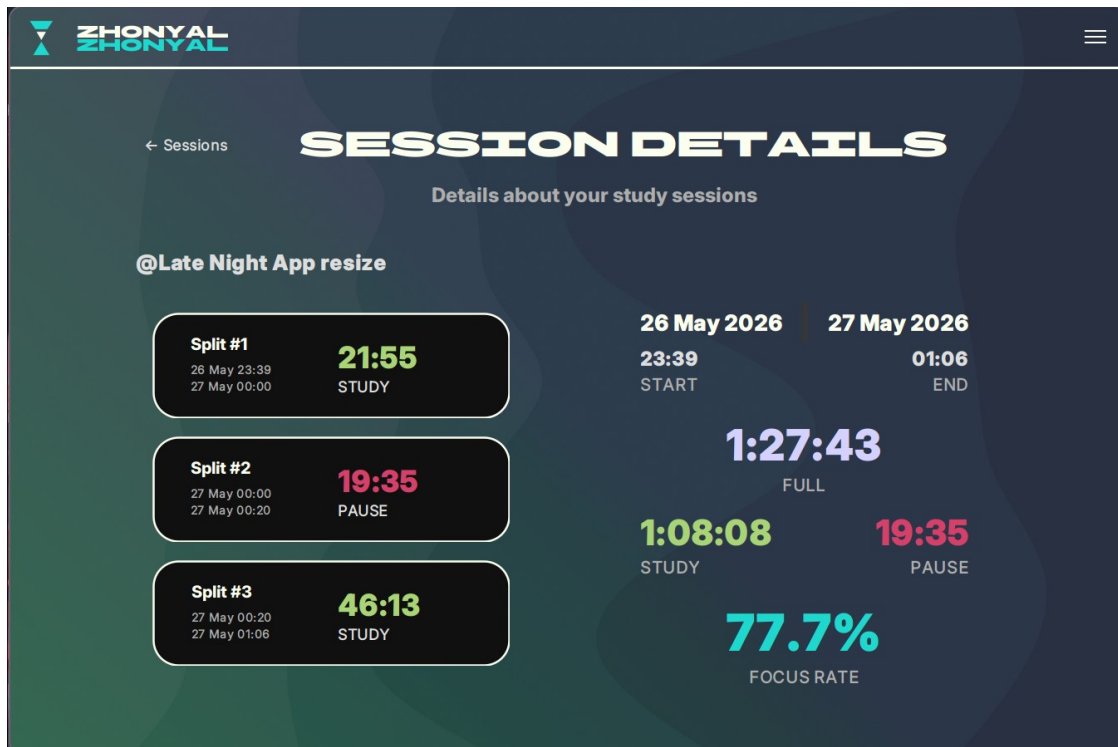


Figura 10. Detalle de una sesión, con sus splits y el focus rate.

Notas

Biblioteca de apuntes en una rejilla de tarjetas con nombre y tamaño de cada archivo. Permite añadir nuevos PDF y activar un modo de borrado.

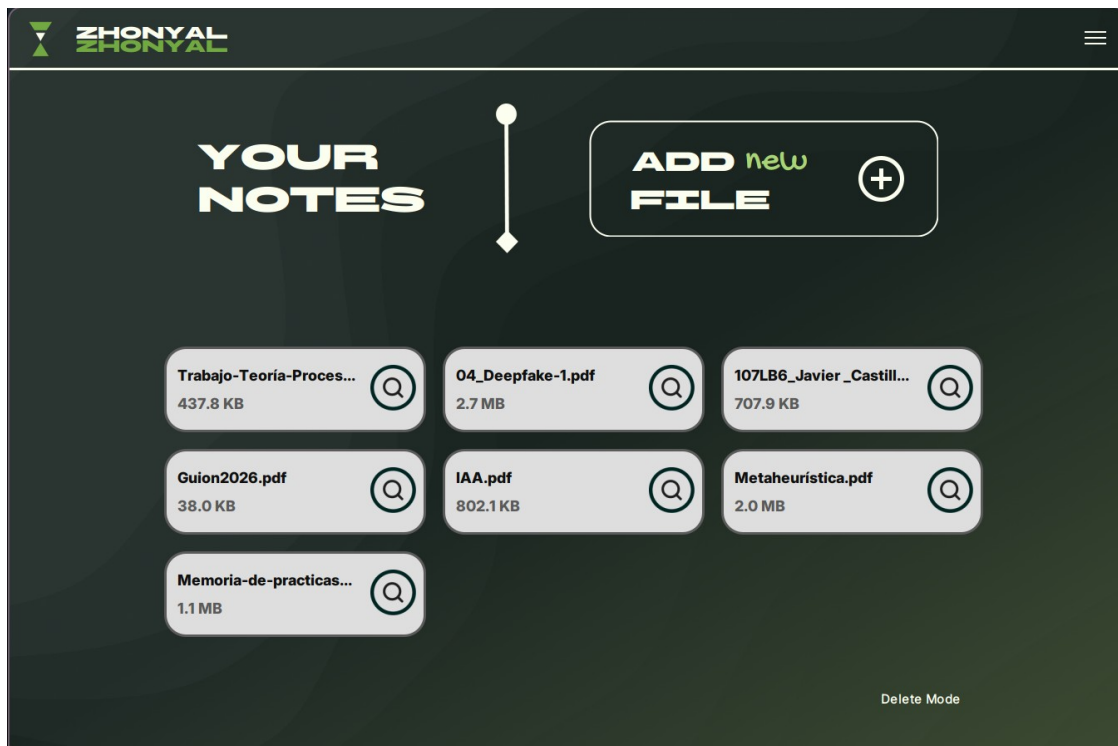


Figura 11. Sección de Notas.

Visor de documentos

Visor integrado en dos paneles: la lista de apuntes a la izquierda y el documento seleccionado a la derecha, con control de zoom. Permite leer los PDF sin salir de la aplicación.

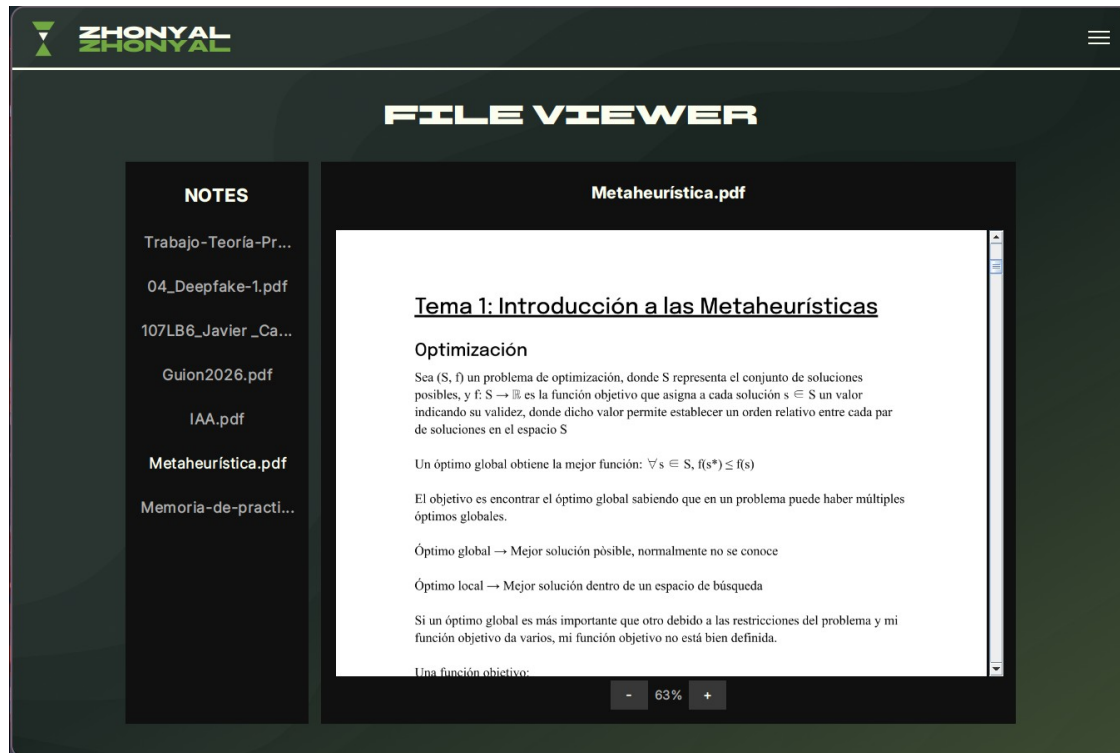


Figura 12. Visor de PDF integrado.

Idioma y menú lateral

El menú lateral da acceso a las secciones (Inicio, Sesiones, Notas), al perfil, al botón de idioma ES/EN y a la opción de limpiar los datos locales. La siguiente captura muestra la interfaz en español, evidenciando la internacionalización.



Figura 13. Interfaz en español con el menú lateral desplegado.

3. Conclusiones

La interfaz final cumple los requisitos planteados: implementa en Java/Swing las pantallas del prototipo, persiste los datos de forma local, integra un visor de PDF y añade un cronómetro flotante fiel a la idea original. Además, incorpora la **internacionalización ES/EN** que en la entrega anterior quedaba pendiente, satisfaciendo el requisito de soportar como mínimo dos idiomas.

He aprendido a utilizar muchísimos conceptos, utilidades nuevas, y al final este es mi primer proyecto hecho en Java. Desconozco las alternativas que hay al día de hoy, pero si Java Swing es la tecnología más usada para este tipo de proyecto entiendo el auge de las aplicaciones web, pudiendo crear un frontend de manera mucho más fácil, atreviéndome a decir que si el diseño lo hubiese podido hacer con HTML y CSS hubiese terminado el proyecto en 2 semanas.

3.1 Cosas que mejorar y añadir

Quiero acabar esta documentación con detalles que me hubiese gustado implementar si hubiese tenido más tiempo a mi disposición, y funcionalidades que añadiré en un futuro, ya que es una aplicación que uso a nivel personal.

- **Atajos de teclado:** Pese a que creo que no es una aplicación tan complicada como para necesitarlo, estos siempre ayudan al usuario, y creo que de cara a iniciar una sesión rápido, o volver atrás, podrían ser útiles
- **Exportar sesiones a otros formatos:** Poder exportar sesiones para tenerlas a mano como pequeños resúmenes
- **Dar opción a crear un archivo de resumen de la sesión:** Poder crear una especie de sesion.md o algo por el estilo, que resuma que se ha abarcado en dicha sesión de estudio.
- **Mandar información por email:** Ya que recopilamos el email en nuestro pseudo login, poder utilizarlo para enviar información si es necesario, o que el usuario pueda mandar sus propias ideas de mejora de la app.
- **Creación de una página de métricas:** Tener una página en la que ver cuanto has trabajado en ciertos periodos de tiempo creo que sería una mejora exponencial en la app.

3.2 Uso de la IA

Pese a que no pone que se deba añadir un apartado sobre esto, quiero añadirlo de cara a la posteridad. Estoy seguro al 100% que tanto yo, como todos mis compañeros han usado IA para esto proyecto, pero quiero dejar claro que la mayoría del trabajo ha sido por mi propia cuenta, para lo bueno y lo malo. Para este proyecto he utilizado Claude Code, con el modelo Opus 4.6. En que apartados me ha ayudado la IA, y en cuales no:

1. Uso de Claude:

- Repetición de líneas de código de diseños que ya había implementado con anterioridad.
- Uso del cardLayout, y explicación de los Runnable

- Diseño de los pop-ups de la app
- Como utilizar Apache PDF Box
- Creación del .exe para Windows (un infierno)
- Creación de la documentación.
- Internacionalización de la aplicación.

2. Manualmente:

- Diseño de la aplicación de manera casi íntegra. (Claude tiene ciertos problemas a la hora de diseñar en Java Swing)
- Diseño de la lógica de las Sesiones, Splits, FilterState y FileUtils.
- Diseño de la BD en sqlite.
- Lógica del guardado de ficheros en /.zhonyal.

El código completo está disponible en el repositorio del proyecto: github.com/JxViii/Zhonyal.

Ha sido un placer, y espero que sean benevolentes con la nota.

— Fin del documento —

Zhonyal · Sistemas Interactivos 2025/26